SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 2 – Pseudocode Writing Task (Algorithm Design)

Course Code:	DSA03	**Course Title:**	Natural Language Processing
CO Assessed:	CO5 – Articulation	**Assessment:**	Assessment Tool 2 – Pseudocode Writing Task (Algorithm Design)
Weightage:	35% of CIA		
CO5 Statement:	Implement semantic models using predicate logic, word sense disambiguation and validate information retrieval techniques. (BL-5)		
Student Name:	Aswanth N N	**Reg. No.:**	192424123
Section / Batch:	2024	**Date:**	26-08-2026

Code of Conduct

I __Aswanth N N_____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: ____Aswanth N N_____

Instructions

- Draw necessary diagrams such as constraint graphs, coreference chains, discourse trees, or predicate logic representations wherever applicable.
- Generate solutions that fully satisfy all given constraints and provide clear evaluation of the output.
- Answers should be concise, well-structured, and supported by evidence from the given text/scenario.
- Duration: 30 minutes.

Section / Topic	Focus	Question No.
Reference Resolution, Discourse	Entity Linking Algorithm	Q1
Text Coherence, Discourse Structure	Coherence Analysis Algorithm	Q2
Dialog Acts, Conversational Agents	Intent Recognition Algorithm	Q3
Language Generation, Surface Realization	NLG Pipeline Design	Q4
Machine Translation, Interlingua, Statistical Approaches	Translation Workflow Algorithm	Q5

Annexure A – Pseudocode Writing Task (Algorithm Design)

1. Reference Resolution Using Multiple Constraints

Design an algorithm to perform **Reference Resolution** by identifying pronouns and selecting the most appropriate antecedent using gender, number, recency, and semantic compatibility.

Apply your algorithm to the discourse:

"Meena gave Kavya a laptop because she needed one for her project. She thanked her and started using it."

Perform the following steps:

1. Identify all entities and referring expressions.
2. Generate possible antecedents for each pronoun.
3. Apply gender, number, recency, and semantic constraints.
4. Select the most appropriate antecedent.
5. Generate the resolved discourse.

Finally, explain how your algorithm avoids incorrect resolution when multiple antecedents have similar grammatical properties.

Ans:

Algorithm: ResolveReferences(discourse)

ALGORITHM ResolveReferences(Discourse D)

INPUT: A sequence of sentences D = {S1, S2, ..., Sn}

OUTPUT: Coreference chain for each pronoun

STEP 1: Entity & Mention Extraction

```
entities ← empty list
FOR each sentence S in D:
    FOR each noun phrase NP in S:
        IF NP is a proper noun or definite NP:
            ADD NP to entities with attributes {gender, number, position, grammatical_role}
    FOR each pronoun P in S:
        ADD P to pronoun_list with attributes {gender, number, position}
```

STEP 2: Candidate Generation

```
FOR each pronoun P in pronoun_list:
    candidates(P) ← all entities E in entities WHERE E.position < P.position
```

```
    // only entities introduced before the pronoun (anaphoric window)

STEP 3: Constraint Filtering
  FOR each pronoun P:
    FOR each candidate C in candidates(P):
      IF C.gender ≠ P.gender: REMOVE C        // Gender agreement
      IF C.number ≠ P.number: REMOVE C        // Number agreement
      IF NOT SemanticCompatible(C, P, context): REMOVE C  // Semantic fit

STEP 4: Scoring and Ranking (for remaining candidates)
  FOR each remaining candidate C:
    score(C) ← 0
    score(C) += RecencyWeight(C.position, P.position)     // closer = higher
    score(C) += RoleWeight(C.grammatical_role)            // subject > object
    score(C) += SemanticFitWeight(C, P, verb_context)     // selectional fit

STEP 5: Selection
  antecedent(P) ← candidate C with MAX(score(C))
  ADD (P → antecedent(P)) to coreference_chain

STEP 6: RETURN coreference_chain
```

Applying to: ***"Meena gave Kavya a laptop because she needed one for her project. She thanked her and started using it."***

Step 1 — Entities & Referring Expressions

Entity/Pronoun	**Type**	**Gender**	**Number**
Meena	Proper noun (subject, S1)	Female	Singular
Kavya	Proper noun (object, S1)	Female	Singular
a laptop	Noun phrase	Neuter	Singular
she (S1)	Pronoun	Female	Singular
one (S1)	Pronoun (indefinite, refers to "laptop" type)	Neuter	Singular
her (S1, "her project")	Pronoun	Female	Singular
She (S2)	Pronoun	Female	Singular
her (S2, "thanked her")	Pronoun	Female	Singular
it (S2)	Pronoun	Neuter	Singular

Step 2 — Candidate Generation

- she(S1) → {Meena, Kavya}
- one(S1) → {a laptop}
- her(S1) → {Meena, Kavya}
- She(S2) → {Meena, Kavya}
- her(S2) → {Meena, Kavya}
- it(S2) → {a laptop}

Step 3 & 4 — Constraint Application + Scoring

Pronoun	**Candidates**	**Gender/Number filter**	**Grammatical role**	**Semantic compatibility**	**Selected**
she (S1)	Meena, Kavya	both pass (F, sg)	Kavya = recent object; but clause "because she needed one" causally follows from Kavya *receiving* a laptop	needing something follows the receiver, not the giver	**Kavya**
one (S1)	a laptop	N/A	—	"one" = an indefinite instance of the same type as "laptop"	**a laptop (type)**
her (S1, project)	Meena, Kavya	both pass	Kavya is current clause subject (she)	project belongs to the person needing the laptop	**Kavya**
She (S2)	Meena, Kavya	both pass	**Recency**: Kavya most recently mentioned (in S1's subordinate clause) BUT subject of S1 main clause was Meena	"thanked her" — thanking implies gratitude toward the giver	**Kavya** (subject continuity — Kavya remains topic from S1's second clause)

her (S2, thanked)	Meena, Kavya	both pass	object position	the one being thanked should be the giver	**Meena**
it (S2)	a laptop	neuter matches	—	"started using it" — you use a laptop, not a project	**a laptop**

Step 5 — Final Antecedent Selection (Resolved)

- she(S1) = Kavya
- one(S1) = a laptop (indefinite reference, same type)
- her(S1) = Kavya
- She(S2) = Kavya
- her(S2) = Meena
- it(S2) = a laptop

Step 6 — Resolved Discourse

"Meena gave Kavya a laptop because **Kavya** needed **a laptop** for **Kavya's** project. **Kavya** thanked **Meena** and started using **the laptop**."

Avoiding Incorrect Resolution with Similar Grammatical Properties

Meena and Kavya share identical **gender** and **number**, so those constraints alone cannot disambiguate — this is exactly the scenario the algorithm is built to handle:

- It does **not rely on a single constraint**; it combines gender/number filtering (Step 3, a hard filter) with **weighted scoring** (Step 4) using recency, grammatical role, and semantic/selectional compatibility.
- **Semantic compatibility acts as the deciding factor** when grammatical properties tie: verbs like "needed," "thanked," and "using" impose selectional restrictions on their arguments (e.g., the receiver needs the item, the receiver thanks the giver), which breaks the tie even though gender/number cannot.
- **Grammatical role parallelism** (subject-to-subject preference, per Centering Theory) is used as an additional scoring signal, favoring continuity of the current discourse topic across clauses.
- By using **multiple independent constraints in combination** rather than any one in isolation, the algorithm avoids the failure mode where two candidates are indistinguishable on a single dimension.

---

2. Algorithm for Entity Chains and Discourse Coherence

Design an algorithm to identify **entity chains** in a discourse and use them to evaluate text coherence.

Apply your algorithm to the following text:

"Anjali bought a new laptop. The laptop had a powerful processor. She installed Python on it. Later, Anjali used the computer to develop a machine learning application."

Perform the following steps:

1. Identify all entities and referring expressions.
2. Link expressions referring to the same entity.
3. Construct entity chains.
4. Calculate or estimate the continuity of entities across sentences.
5. Determine whether the discourse is coherent.

Finally, explain how breaking an entity chain could reduce discourse coherence.

Ans:

Algorithm: BuildEntityChains(discourse)

```
ALGORITHM BuildEntityChains(Discourse D)
INPUT: Sequence of sentences D = {S1, ..., Sn}
OUTPUT: Entity chains + coherence score

STEP 1: Mention Extraction
   mentions ← empty list
   FOR each sentence S in D:
      FOR each NP or pronoun M in S:
         ADD M to mentions with {surface_form, position, sentence_index, type}

STEP 2: Mention Linking
   chains ← empty list
   FOR each mention M in mentions (in order):
      best_chain ← NULL
      FOR each existing chain C in chains:
         IF IsCoreferent(M, LastMention(C)):   // via string match, synonym, pronoun resolution
            best_chain ← C
            BREAK
      IF best_chain ≠ NULL:
         ADD M to best_chain
      ELSE:
         CREATE new chain with M
         ADD new chain to chains

STEP 3: Chain Construction Output
   FOR each chain C in chains:
      RECORD entity_name(C), mentions(C), sentence_span(C)

STEP 4: Continuity Estimation
   FOR each chain C:
      continuity(C) ← (number of sentences containing a mention of C) / (total sentences n)
   overall_continuity ← AVERAGE(continuity(C) for all chains C)

STEP 5: Coherence Determination
```

IF overall_continuity ≥ threshold (e.g., 0.5) AND every sentence has ≥1 chain mention:

discourse ← "Coherent"

ELSE:

discourse ← "Low coherence" — flag sentences with no chain overlap

STEP 6: RETURN chains, overall_continuity, coherence_verdict

Applying to: ***"Anjali bought a new laptop. The laptop had a powerful processor. She installed Python on it. Later, Anjali used the computer to develop a machine learning application."***

Step 1 — Entities/Mentions

Sentence	**Mentions**
S1	Anjali, a new laptop
S2	The laptop, a powerful processor
S3	She, it
S4	Anjali, the computer, a machine learning application

Step 2 & 3 — Entity Chains

- **Chain A (Anjali):** Anjali(S1) → She(S3) → Anjali(S4)
- **Chain B (laptop):** a new laptop(S1) → The laptop(S2) → it(S3) → the computer(S4)
- **Chain C (processor):** a powerful processor(S2) — single mention, no continuation
- **Chain D (ML application):** a machine learning application(S4) — single mention (new entity)

Step 4 — Continuity Estimation (n = 4 sentences)

Chain	**Sentences present**	**Continuity**
A (Anjali)	S1, S3, S4	3/4 = 0.75
B (laptop)	S1, S2, S3, S4	4/4 = 1.0
C (processor)	S2	1/4 = 0.25
D (ML application)	S4	1/4 = 0.25

Overall average continuity ≈ (0.75+1.0+0.25+0.25)/4 = **0.5625**

Step 5 — Coherence Determination

- Every sentence contains at least one mention from Chain A or B → **discourse is coherent**.
- Chain B (laptop → computer) has **perfect continuity (1.0)**, acting as the backbone entity linking all four sentences.
- Chains C and D are single-mention "satellite" entities (elaborative details), which is normal and doesn't harm coherence since the main entity chains (Anjali, laptop) persist throughout.

Verdict: Coherent discourse, anchored by two strong entity chains (Anjali, laptop/computer) with continuous topic progression.

How Breaking an Entity Chain Reduces Coherence

- If, e.g., S3 had said "She installed Python on **the tablet**" instead of "it," the laptop chain would break — introducing an entity with **no established link**, forcing the reader to either assume it's an error or a brand-new unexplained object.
- A broken chain causes **loss of topic continuity**: the discourse would appear to jump between unrelated entities rather than progressively elaborating on one.
- It also causes **referential failure** in coreference systems — pronouns like "it" would have no valid antecedent, degrading automatic understanding (e.g., in summarization or QA over the text).
- Coherence models (e.g., Centering Theory) predict that abrupt shifts in the "backward-looking center" (Cb) between sentences — caused by broken chains — lead to perceived **incoherence or "rough shifts,"** making the text harder to follow even if grammatically correct.

---

3. Algorithm for Identifying Discourse Relations

Design an algorithm to identify **discourse relations** between consecutive sentences using discourse markers and contextual meaning.

Apply your algorithm to:

"The server crashed unexpectedly. As a result, users could not access the application. The technical team restarted the server. After that, the system became available again."

Your algorithm should identify relations such as:

- Cause–Effect
- Sequence
- Problem–Solution
- Elaboration

Perform the following steps:

1. Divide the discourse into individual discourse units.
2. Identify explicit discourse markers.
3. Analyze the semantic relationship between units.
4. Assign an appropriate discourse relation.
5. Construct a discourse structure representation.

Finally, justify how the identified relations make the discourse logically coherent.

Ans:

Algorithm: IdentifyDiscourseRelations(discourse)

ALGORITHM IdentifyDiscourseRelations(Discourse D)

```
INPUT: Discourse D split into discourse units (clauses/sentences)
OUTPUT: List of discourse relations between consecutive units

STEP 1: Segmentation
  units ← SplitIntoDiscourseUnits(D)   // typically clause- or sentence-level

STEP 2: Discourse Marker Detection
  marker_table ← {
    "because", "as a result", "so", "due to"   → Cause-Effect
    "then", "after that", "later", "next"      → Sequence
    "however", "but"                           → Contrast
    "in addition", "such as", "for example"    → Elaboration
    "to fix this", "the solution was", "restarted" (domain-specific) → Problem-Solution
  }
  FOR each pair of consecutive units (Ui, Ui+1):
    marker ← ExtractMarker(Ui+1, marker_table)

STEP 3: Semantic Relationship Analysis
  FOR each pair (Ui, Ui+1):
    IF marker found:
      relation ← marker_table[marker]
    ELSE:
      relation ← InferFromSemantics(Ui, Ui+1)
      // e.g., check verb semantics: failure-verb followed by repair-verb → Problem-Solution
      // event Ui+1 temporally follows Ui with no causal marker → Sequence
      // Ui+1 restates/adds detail about same entity as Ui → Elaboration

STEP 4: Relation Assignment
  FOR each pair (Ui, Ui+1):
    ASSIGN relation(Ui, Ui+1) ← relation
    ADD (Ui, relation, Ui+1) to discourse_structure

STEP 5: Discourse Structure Construction
  BUILD a directed graph / tree:
    nodes = discourse units
    edges = labeled relations between consecutive/related units
```

STEP 6: RETURN discourse_structure

Applying to: ***"The server crashed unexpectedly. As a result, users could not access the application. The technical team restarted the server. After that, the system became available again."***

Step 1 — Discourse Units

- U1: "The server crashed unexpectedly."
- U2: "As a result, users could not access the application."
- U3: "The technical team restarted the server."
- U4: "After that, the system became available again."

Step 2 — Marker Detection

Unit pair	Marker found	Marker type
U1 → U2	"As a result"	Cause-Effect marker
U2 → U3	none explicit	—
U3 → U4	"After that"	Sequence marker

Step 3 & 4 — Semantic Analysis & Relation Assignment

Unit Pair	**Relation**	**Justification**
U1 → U2	**Cause–Effect**	Explicit marker "as a result"; server crash directly causes access failure
U2 → U3	**Problem–Solution**	No explicit marker, but semantically: U1–U2 describe a problem (crash, inaccessibility); U3 ("restarted the server") is an action that resolves it — inferred via domain/action semantics (failure-verb → repair-verb pattern)
U3 → U4	**Sequence / Cause-Effect**	"After that" signals temporal sequence, but restarting *causing* availability is also a causal link — this is a **Sequence relation with an underlying causal effect** (temporal marker takes surface priority, so labeled Sequence)

Step 5 — Discourse Structure Representation

[U1: Server crashed]

│ Cause-Effect

▼

[U2: Users could not access application]

│ Problem-Solution

▼

[U3: Technical team restarted server]

│ Sequence (with causal effect)

▼

[U4: System became available again]

Overall discourse type: **Problem–Solution narrative**, with an internal Cause-Effect (U1–U2) and Sequence (U3–U4) sub-structure.

Justification: Why These Relations Make the Discourse Logically Coherent

- The **Cause-Effect** link (U1→U2) explains *why* the problem occurred, giving the reader a logical reason for the consequence rather than two disconnected facts.
- The **Problem-Solution** link (U2→U3) connects the negative event to the corrective action, showing narrative purpose — the team's action directly addresses the stated problem, not a random unrelated event.
- The **Sequence/causal** link (U3→U4) shows the resolution's outcome, closing the narrative loop (problem → action → resolution).
- Together, these relations form a coherent **cause → effect → response → outcome** chain, which mirrors real-world causal reasoning and lets the reader track *why* each event happens, not just *that* it happens — this is the core function of discourse relations in producing coherent (versus merely grammatical) text.

---

4. Algorithm for Dialogue Act Classification

Design an algorithm to recognize **Dialogue Acts** in a conversation.

Apply your algorithm to:

User: "Hello, I need help with my assignment."

Agent: "Sure, what problem are you facing?"

User: "I do not understand machine learning."

Agent: "I can explain the basic concepts to you."

Classify each utterance into one of the following:

- Greeting
- Request
- Question
- Inform
- Offer
- Confirmation

Perform the following steps:

1. Preprocess each utterance.
2. Extract relevant linguistic features.
3. Identify the communicative intention.
4. Assign a dialogue act.

5. Generate the dialogue-act sequence.

Finally, explain how dialogue-act recognition helps a conversational agent select an appropriate next response.

Ans:

Algorithm: ClassifyDialogueAct(utterance)

ALGORITHM ClassifyDialogueActs(Conversation Conv)

INPUT: Sequence of utterances U1, U2, ..., Un with speaker labels

OUTPUT: Dialogue act label for each utterance

STEP 1: Preprocessing

```
FOR each utterance U:
    tokens ← Tokenize(U)
    normalize(tokens)   // lowercase, remove punctuation noise (keep "?" as feature)
```

STEP 2: Feature Extraction

```
FOR each utterance U:
    features(U) ← {
        has_greeting_word: U contains "hello", "hi", "hey"
        has_question_mark: U ends with "?"
        has_wh_word: U contains "what", "where", "how", "why"
        has_request_verb: U contains "please", "can you", "I need", "help"
        has_modal_offer: U contains "can", "I can", "let me", "shall I"
        has_declarative_info: U contains factual/statement structure with no "?"
        has_confirmation_word: U contains "yes", "sure", "okay", "got it"
    }
```

STEP 3: Intention Identification (rule-based decision list)

```
FOR each utterance U:
    IF features(U).has_greeting_word: intent ← Greeting
    ELIF features(U).has_request_verb AND NOT has_question_mark: intent ← Request
    ELIF features(U).has_question_mark AND has_wh_word: intent ← Question
    ELIF features(U).has_modal_offer AND speaker = Agent: intent ← Offer
    ELIF features(U).has_confirmation_word: intent ← Confirmation
    ELIF features(U).has_declarative_info: intent ← Inform
    ELSE: intent ← Inform (default fallback)
```

STEP 4: Dialogue Act Assignment

label(U) ← intent

STEP 5: Sequence Generation

dialogue_act_sequence ← [label(U1), label(U2), ..., label(Un)]

STEP 6: RETURN dialogue_act_sequence

Applying to the Conversation

Utterance	**Features detected**	**Assigned Dialogue Act**
User: "Hello, I need help with my assignment."	greeting word "Hello" (dominant first feature)	**Greeting** *(with embedded Request — "I need help" — but Greeting takes priority as opening act)*
Agent: "Sure, what problem are you facing?"	"Sure" = confirmation-like opener, but "what...?" (wh + question mark) dominates	**Question**
User: "I do not understand machine learning."	declarative, no question mark, states a fact/problem	**Inform**
Agent: "I can explain the basic concepts to you."	modal offer "I can", agent speaker	**Offer**

Dialogue Act Sequence: [Greeting, Question, Inform, Offer]

(Note: In finer-grained schemes, "Hello, I need help" is sometimes split into two acts — Greeting + Request — since it contains both a salutation and an implicit request for assistance. If splitting is required: [Greeting+Request, Question, Inform, Offer].)

How Dialogue Act Recognition Helps Response Selection

- **Maps intent to response type:** Once an utterance is classified (e.g., "Question"), the agent knows it must generate an **Answer**-type act next, not another Question or a Greeting — this narrows the response-generation search space.
- **Enables dialogue policy rules:** A conversational agent can use a simple mapping (Greeting→Greeting-back, Request→Offer/Acknowledge, Question→Answer, Inform→Acknowledge+FollowUp) to select contextually appropriate next actions rather than generating free-form, potentially irrelevant text.

- **Supports dialogue state tracking:** Recognizing "Inform" acts (e.g., the user stating they don't understand ML) tells the system to update its model of user needs and choose an **Offer** (explain concepts) as the logical next act — directly seen in this conversation.
- **Improves coherence and task success:** By aligning each response's act with the preceding act (adjacency pairs, e.g., Question→Answer, Offer→Accept/Decline), the conversation stays on-task and avoids non-sequiturs, which is critical for task-oriented agents (e.g., tutoring, booking systems).

---

5. Algorithm for Dialogue State Tracking

Design an algorithm for a **Conversational Agent** that maintains dialogue state while collecting information from a user.

Apply your algorithm to:

User: "I want to book a flight."

Agent: "Where would you like to travel?"

User: "I want to go to Delhi."

Agent: "From which city?"

User: "From Chennai."

Your algorithm should maintain the following slots:

Departure City

Destination City

Travel Date

Number of Passengers

Perform the following steps:

1. Initialize the dialogue state.
2. Identify information provided in each utterance.
3. Update the corresponding slot.
4. Identify missing information.
5. Generate the next appropriate question.

Finally, explain how dialogue state tracking improves coherence in a multi-turn conversation.

Ans:

Algorithm: TrackDialogueState(conversation)

ALGORITHM TrackDialogueState(Conversation Conv)

INPUT: Sequence of user utterances U1, U2, ..., Un

```
OUTPUT: Filled slot-value dialogue state + next system question

STEP 1: Initialize Dialogue State
  state ← {
    DepartureCity: NULL,
    DestinationCity: NULL,
    TravelDate: NULL,
    NumPassengers: NULL
  }
  slot_order ← [DestinationCity, DepartureCity, TravelDate, NumPassengers]
  // order can be adapted dynamically based on what user volunteers first

STEP 2: Utterance Processing (per turn)
  FOR each user utterance U:
    entities ← ExtractEntities(U)   // NER: city names, dates, numbers
    intents ← DetectIntent(U)       // e.g., "book_flight", "provide_info"

STEP 3: Slot Update
    FOR each extracted entity E in entities:
      IF E.type = CITY:
        IF U contains pattern "to <city>" OR context expects destination:
          state.DestinationCity ← E.value
        ELIF U contains pattern "from <city>" OR context expects departure:
          state.DepartureCity ← E.value
      ELIF E.type = DATE:
        state.TravelDate ← E.value
      ELIF E.type = NUMBER AND context = passenger_count:
        state.NumPassengers ← E.value

STEP 4: Missing Slot Identification
    missing_slots ← [slot for slot in state WHERE state[slot] = NULL]

STEP 5: Next Question Generation
    IF missing_slots is empty:
      GENERATE confirmation summary using filled state
    ELSE:
```

next_slot ← FIRST(missing_slots) // or priority-ordered

GENERATE question corresponding to next_slot

// e.g., DepartureCity → "From which city?"

STEP 6: RETURN updated state, next_question

Applying to the Conversation

Turn 1:

- User: "I want to book a flight." → intent = book_flight; no slots filled.
- State: {Departure: NULL, Destination: NULL, Date: NULL, Passengers: NULL}
- Missing: all 4 slots → Agent asks for **Destination** first (common flow priority).
- Agent: "Where would you like to travel?"

Turn 2:

- User: "I want to go to Delhi." → entity extracted: Delhi (CITY), pattern "go to X" → destination context.
- **Update:** DestinationCity ← "Delhi"
- State: {Departure: NULL, Destination: "Delhi", Date: NULL, Passengers: NULL}
- Missing: Departure, Date, Passengers → next priority = Departure.
- Agent: "From which city?"

Turn 3:

- User: "From Chennai." → entity extracted: Chennai (CITY), pattern "from X" → departure context.
- **Update:** DepartureCity ← "Chennai"
- State: {Departure: "Chennai", Destination: "Delhi", Date: NULL, Passengers: NULL}
- Missing: Date, Passengers → next priority = Date.
- Agent (next expected question): "On what date would you like to travel?"

Final State Table (after 3 turns):

Slot	**Value**
Departure City	Chennai
Destination City	Delhi
Travel Date	*(not yet provided)*
Number of Passengers	*(not yet provided)*

How Dialogue State Tracking Improves Multi-Turn Coherence

- **Persistent memory across turns:** DST maintains a structured record of everything the user has said so far, so the agent never asks for information already given (e.g., it won't re-ask "where would you like to travel?" after Delhi is known) — this avoids repetitive, incoherent exchanges.

- **Grounds pronoun/ellipsis resolution:** Later utterances like "change it to next Monday" can be resolved against the state's TravelDate slot, since DST provides the necessary context that a stateless system would lack.
- **Guides principled next-question generation:** By tracking which slots are filled vs. missing, the system can generate contextually appropriate, non-redundant questions in a logical order — creating a natural, task-driven conversation flow instead of jumping randomly between topics.
- **Enables error recovery and updates:** If the user later says "actually, make it Bangalore," DST allows the system to correctly update just the relevant slot (Destination) without disrupting the rest of the collected information — preserving overall dialogue coherence even amid corrections.
- **Supports task completion tracking:** DST enables the system to know exactly when all required information has been gathered (all slots filled) and it's time to move from information-gathering to confirmation/booking — a critical coherence signal for task-oriented dialogue systems.

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient

Q. No. / Criteria Level	Problem Understanding	Algorithm Design	Use of Control Structures	Pseudocode Structure	Completeness	Sub. Total	Total %
1	4	3	4	4	4	19	93
2	4	4	4	4	3	19	
3	4	4	4	4	3	19	

4	4	3	4	4	4	19	
5	4	3	4	4	4	19	

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____